Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-213БВ-25

Студент: Крайнова О.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.11.26

Москва, 2026

Постановка задачи

Вариант 13.

Наложить K раз фильтр, использующий матрицу свертки, на матрицу, состоящую из вещественных чисел. Размер окна задается пользователем.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы.

Общий метод и алгоритм решения

Использованные системные вызовы и функции POSIX Threads:

- int clock_gettime(clockid_t clk_id, struct timespec *tp) - получение текущего времени.
- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg) - создание рабочего потока.
- int pthread_join(pthread_t thread, void **retval) - ожидание завершения потока.
- int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned int count) - инициализация барьера.
- int pthread_barrier_wait(pthread_barrier_t *barrier) - блокировка потока на барьере до тех пор, пока все потоки не достигнут точки синхронизации.
- int pthread_barrier_destroy(pthread_barrier_t *barrier) - уничтожение барьера после завершения параллельных вычислений.

Программа запускается в виде:

./lab2 --size <число> --window <число> --iterations <число> --threads <число>

Параметры задаются ключами запуска и проверяются на корректность:

--size – размер матрицы вещественных чисел (от 1 до 10000);
--window – размер матрицы свертки (от 1 до 11, строго меньше размера матрицы);
--iterations – количество раз, которое надо наложить матрицу свертки (от 1 до 10000);
--threads – количество потоков (от 1 до 10000).

Программа выполняет многократное наложение двумерного фильтра (свёртку) на квадратную матрицу. Последовательная версия обрабатывает всю матрицу в одном потоке. Параллельная версия распределяет строки матрицы между потоками равномерно. После завершения каждой итерации все потоки синхронизируются на барьере, чтобы гарантировать, что чтение из матрицы следующей итерации происходит только после записи всех результатов. Главный поток ожидает завершения рабочих через pthread_join. Время измеряется через CLOCK_MONOTONIC.

Для оценки параллельной версии используются ускорение S = T_seq / T_par и эффективность E = S / p, где T_seq и T_par — время последовательной и параллельной версий соответственно, а p - число потоков. Ускорение показывает, во сколько раз параллельная версия быстрее последовательной, а эффективность показывает, насколько рационально используются вычислительные ресурсы. Верхнюю границу ускорения задаёт закон Амдала: чем больше в программе последовательного кода, тем меньше потенциальный выигрыш от параллельности.

Код программы

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <pthread.h>


typedef struct ThreadArgs {
    double *source;
    double *dest;
    double *filter;

    int size;
    int window;
    int iterations;

    int thread_id;
    int num_threads;

    pthread_barrier_t *barrier;
} ThreadArgs;


double get_pixel(double *matrix, int size, int i, int j) {
    if (i < 0 || i >= size || j < 0 || j >= size) {
        return 0.0;
    }
    return matrix[i * size + j];
}


double *use_filter(double *source, double *dest, double *filter, int size, int window, int iterations)
{
    int half_w = window / 2;

    for (int iter = 0; iter < iterations; iter++) {

        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                double sum = 0.0;

                for (int di = -half_w; di <= half_w; di++) {
                    for (int dj = -half_w; dj <= half_w; dj++) {
                        double pixel = get_pixel(source, size, i + di, j + dj);
                        int flt_idx = (di + half_w) * window + (dj + half_w);
```

```
                sum += pixel * filter[flt_idx];
            }
        }

        dest[i * size + j] = sum;
      }
    }

    double *temp = source;
    source = dest;
    dest = temp;
  }
  return source;
}


void *thread_worker(void *arg) {
  ThreadArgs *args = (ThreadArgs *)arg;

  int size = args->size;
  int window = args->window;
  int iterations = args->iterations;
  int thread_id = args->thread_id;
  int num_threads = args->num_threads;

  int half_w = window / 2;

  int start_row = (size * thread_id) / num_threads;
  int end_row = (size * (thread_id + 1)) / num_threads;

  double *curr_source = args->source;
  double *curr_dest = args->dest;
  double *curr_filter = args->filter;

  for (int iter = 0; iter < iterations; iter++) {
    for (int i = start_row; i < end_row; i++) {
      for (int j = 0; j < size; j++) {
        double sum = 0.0;

        for (int di = -half_w; di <= half_w; di++) {
          for (int dj = -half_w; dj <= half_w; dj++) {
            double pixel = get_pixel(curr_source, size, i + di, j + dj);
            int flt_idx = (di + half_w) * window + (dj + half_w);
            sum += pixel * curr_filter[flt_idx];
          }
        }
        curr_dest[i * size + j] = sum;
```

```
            }
        }

        pthread_barrier_wait(args->barrier);

        double *temp = curr_source;
        curr_source = curr_dest;
        curr_dest = temp;
    }

    return curr_source;
}

double *parallel_use_filter(double *source, double *dest, double *filter, int size, int window, int iterations, int num_threads) {
    pthread_t *threads = malloc((size_t)num_threads * sizeof(pthread_t));
    ThreadArgs *args = malloc((size_t)num_threads * sizeof(ThreadArgs));

    if (threads == NULL || args == NULL) {
        printf("Ошибка выделения памяти для потоков\n");
        exit(1);
    }

    pthread_barrier_t barrier;
    pthread_barrier_init(&barrier, NULL, num_threads);

    for (int t = 0; t < num_threads; t++) {
        args[t].source = source;
        args[t].dest = dest;
        args[t].filter = filter;
        args[t].size = size;
        args[t].window = window;
        args[t].iterations = iterations;
        args[t].thread_id = t;
        args[t].num_threads = num_threads;
        args[t].barrier = &barrier;

        if (pthread_create(&threads[t], NULL, thread_worker, &args[t]) != 0) {
            printf("Ошибка создания потока %d\n", t);
            exit(1);
        }
    }

    void *result;
    pthread_join(threads[0], &result);

    for (int t = 1; t < num_threads; t++) {
        pthread_join(threads[t], NULL);
```

```
    }

    pthread_barrier_destroy(&barrier);
    free(threads);
    free(args);

    return (double *)result;
}


int main(int argc, char *argv[]) {
    int size = 100;
    int window = 3;
    int iterations = 10;
    int threads = 1;

    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--size") == 0 && i + 1 < argc) {
            size = atoi(argv[i + 1]);
            i++;
        }
        else if (strcmp(argv[i], "--window") == 0 && i + 1 < argc) {
            window = atoi(argv[i + 1]);
            i++;
        }
        else if (strcmp(argv[i], "--iterations") == 0 && i + 1 < argc) {
            iterations = atoi(argv[i + 1]);
            i++;
        }
        else if (strcmp(argv[i], "--threads") == 0 && i + 1 < argc) {
            threads = atoi(argv[i + 1]);
            i++;
        }
        else {
            printf("Неизвестный аргумент: %s\n", argv[i]);
            return 1;
        }
    }

    if (size <= 0 || size > 10000) {
        printf("Размер матрицы должен быть от 1 до 10000\n");
        return 1;
    }

    if (window <= 0 || window % 2 == 0 || window > 11) {
        printf("Размер фильтра должен быть нечётным числом от 1 до 11\n");
        return 1;
    }
```

```
if (window >= size) {
    printf("Размер фильтра должен быть строго меньше размера матрицы\n");
    return 1;
}

if (iterations <= 0 || iterations > 10000) {
    printf("Количество итераций должно быть от 1 до 10000\n");
    return 1;
}

if (threads <= 0 || threads > 10000) {
    printf("Количество потоков должно быть от 1 до 10000\n");
    return 1;
}


double *source = (double *)malloc((size_t)size * size * sizeof(double));
double *dest = (double *)malloc((size_t)size * size * sizeof(double));

if (source == NULL || dest == NULL) {
    printf("Не удалось выделить память под матрицы\n");
    if (source) free(source);
    if (dest) free(dest);
    return 1;
}

double *filter = (double *)malloc((size_t)window * window * sizeof(double));
if (filter == NULL) {
    printf("Не удалось выделить память под фильтр\n");
    free(source);
    free(dest);
    return 1;
}

srand(42);
for (int i = 0; i < size * size; i++) {
    source[i] = (double)rand() / RAND_MAX;
}

double filter_value = 1.0 / (window * window);
for (int i = 0; i < window * window; i++) {
    filter[i] = filter_value;
}

if (size <= 5) {
    printf("Исходная матрица (%dx%d)\n", size, size);
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
```

```
            printf("%.4f\t", source[i * size + j]);
        }
        printf("\n");
    }

    printf("Фильтр (%dx%d)\n", window, window);
    for (int i = 0; i < window; i++) {
        for (int j = 0; j < window; j++) {
            printf("%.4f\t", filter[i * window + j]);
        }
        printf("\n");
    }
}

struct timespec start_seq, end_seq;
clock_gettime(CLOCK_MONOTONIC, &start_seq);

double *result = use_filter(source, dest, filter, size, window, iterations);

clock_gettime(CLOCK_MONOTONIC, &end_seq);

double time_seq = (end_seq.tv_sec - start_seq.tv_sec) * 1000.0 + (end_seq.tv_nsec -
start_seq.tv_nsec) / 1000000.0;

printf("Последовательная версия: %.3f миллисекунд\n", time_seq);

if (size <= 5) {
    printf("Итоговая матрица после последовательной свертки (%dx%d)\n", size, size);
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            printf("%.4f\t", result[i * size + j]);
        }
        printf("\n");
    }
}


printf("\n");
srand(42);
for (int i = 0; i < size * size; i++) {
    source[i] = (double)rand() / RAND_MAX;
}

struct timespec start_par, end_par;
clock_gettime(CLOCK_MONOTONIC, &start_par);

double *par_result = parallel_use_filter(source, dest, filter, size, window, iterations, threads);
```

```
    clock_gettime(CLOCK_MONOTONIC, &end_par);

    printf("Параллельная версия (%d потоков):\n", threads);
    if (size <= 5) {
        printf("Итоговая матрица после параллельной свертки (%dx%d)\n", size, size);
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                printf("%.4f\t", par_result[i * size + j]);
            }
            printf("\n");
        }
    }

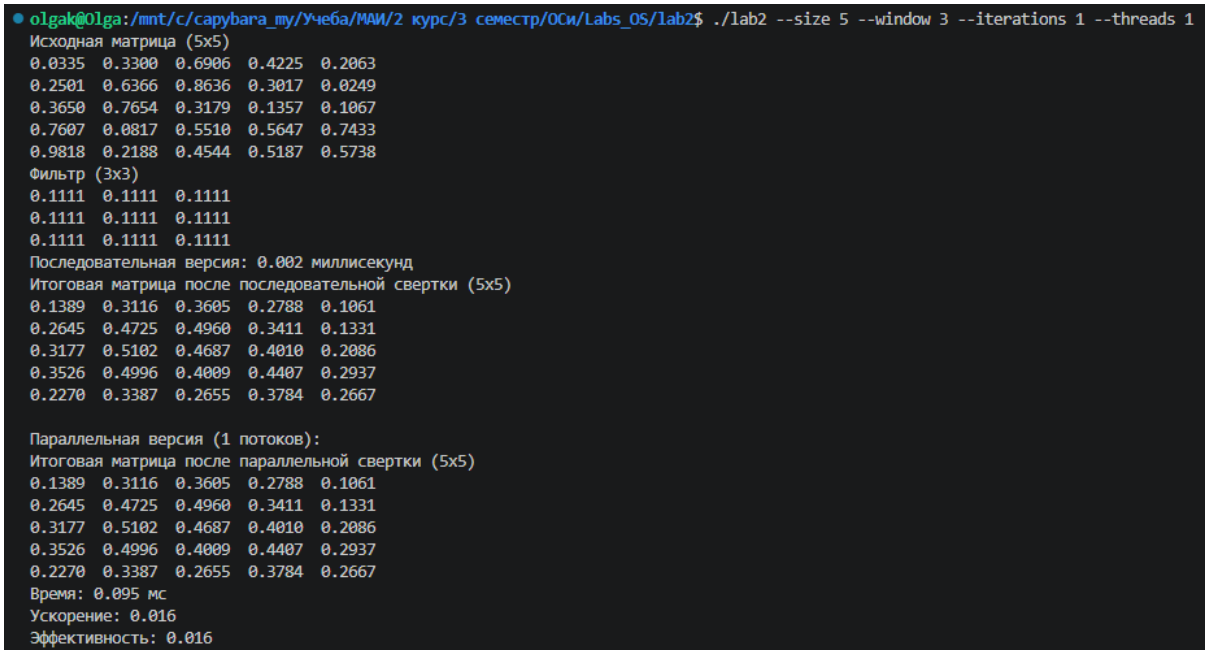
    double time_par = (end_par.tv_sec - start_par.tv_sec) * 1000.0 + (end_par.tv_nsec -
start_par.tv_nsec) / 1000000.0;
    double speedup = time_seq / time_par;
    double efficiency = speedup / threads;

    printf("Время: %.3f мс\n", time_par);
    printf("Ускорение: %.3f\n", speedup);
    printf("Эффективность: %.3f\n", efficiency);

    free(source);
    free(dest);
    free(filter);
    return 0;
}
```

Протокол работы программы

Тесты на маленькой матрице с небольшим количеством итераций:



```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 3 --window 3 --iterations 5 --threads 2
Размер фильтра должен быть строго меньше размера матрицы
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 4 --window 3 --iterations 5 --threads 2
Исходная матрица (4x4)
0.0335  0.3300  0.6906  0.4225
0.2063  0.2501  0.6366  0.8636
0.3017  0.0249  0.3650  0.7654
0.3179  0.1357  0.1067  0.7607
Фильтр (3x3)
0.1111  0.1111  0.1111
0.1111  0.1111  0.1111
0.1111  0.1111  0.1111
Последовательная версия: 0.004 миллисекунд
Итоговая матрица после последовательной свертки (4x4)
0.0479  0.0817  0.0869  0.0563
0.0762  0.1301  0.1384  0.0897
0.0747  0.1275  0.1357  0.0880
0.0454  0.0775  0.0825  0.0535

Параллельная версия (2 потоков):
Итоговая матрица после параллельной свертки (4x4)
0.0479  0.0817  0.0869  0.0563
0.0762  0.1301  0.1384  0.0897
0.0747  0.1275  0.1357  0.0880
0.0454  0.0775  0.0825  0.0535
Время: 0.189 мс
Ускорение: 0.020
Эффективность: 0.010
```

```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 10 --window 5 --iterations 10 --threads 32
Последовательная версия: 0.129 миллисекунд

Параллельная версия (32 потоков):
Время: 4.494 мс
Ускорение: 0.029
Эффективность: 0.001
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 10 --window 5 --iterations 10 --threads 128
Последовательная версия: 0.131 миллисекунд

Параллельная версия (128 потоков):
Время: 19.192 мс
Ускорение: 0.007
Эффективность: 0.000
```

Тесты на большой матрице:

```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 1
Последовательная версия: 2994.822 миллисекунд

Параллельная версия (1 потоков):
Время: 3131.506 мс
Ускорение: 0.956
Эффективность: 0.956
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 2
Последовательная версия: 3063.798 миллисекунд

Параллельная версия (2 потоков):
Время: 1668.716 мс
Ускорение: 1.836
Эффективность: 0.918
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 4
Последовательная версия: 3100.843 миллисекунд

Параллельная версия (4 потоков):
Время: 997.325 мс
Ускорение: 3.109
Эффективность: 0.777
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 8
Последовательная версия: 3016.820 миллисекунд

Параллельная версия (8 потоков):
Время: 820.436 мс
Ускорение: 3.677
Эффективность: 0.460
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 12
Последовательная версия: 2986.596 миллисекунд

Параллельная версия (12 потоков):
Время: 600.015 мс
Ускорение: 4.978
Эффективность: 0.415
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 32
Последовательная версия: 3066.339 миллисекунд

Параллельная версия (32 потоков):
Время: 651.755 мс
Ускорение: 4.705
Эффективность: 0.147
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 128
Последовательная версия: 2976.312 миллисекунд

Параллельная версия (128 потоков):
Время: 653.831 мс
Ускорение: 4.552
Эффективность: 0.036
```

```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 500 --window 5 --iterations 150 --threads 1024
Последовательная версия: 3073.887 миллисекунд

Параллельная версия (1024 потоков):
Время: 1387.327 мс
Ускорение: 2.216
Эффективность: 0.002
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ ./lab2 --size 1000 --window 9 --iterations 150 --threads 128
Последовательная версия: 38558.786 миллисекунд

Параллельная версия (128 потоков):
Время: 7813.267 мс
Ускорение: 4.935
Эффективность: 0.039
```

Потоки при запуске программы:

```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ strace -f -e trace=clone3 ./lab2 --size 100 --win
dow 3 --iterations 5 --threads 4
Последовательная версия: 1.795 миллисекунд

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_
CHILD_CLEARTID, child_tid=0x793c48bff990, parent_tid=0x793c48bff990, exit_signal=0, stack=0x793c483ff000, stack_size=0x7fff8
0, tls=0x793c48bff6c0}strace: Process 10079 attached
 => {parent_tid=[10079]}, 88) = 10079
[pid 10078] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_S
ETTID|CLONE_CHILD_CLEARTID, child_tid=0x793c483fe990, parent_tid=0x793c483fe990, exit_signal=0, stack=0x793c47bfe000, stack_
size=0x7fff80, tls=0x793c483fe6c0}strace: Process 10080 attached
 => {parent_tid=[10080]}, 88) = 10080
[pid 10078] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_S
ETTID|CLONE_CHILD_CLEARTID, child_tid=0x793c47bfd990, parent_tid=0x793c47bfd990, exit_signal=0, stack=0x793c473fd000, stack_
size=0x7fff80, tls=0x793c47bfd6c0}strace: Process 10081 attached
 => {parent_tid=[10081]}, 88) = 10081
[pid 10078] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_S
ETTID|CLONE_CHILD_CLEARTID, child_tid=0x793c473fc990, parent_tid=0x793c473fc990, exit_signal=0, stack=0x793c46bfc000, stack_
size=0x7fff80, tls=0x793c473fc6c0}strace: Process 10082 attached
 => {parent_tid=[10082]}, 88) = 10082
[pid 10082] +++ exited with 0 +++
[pid 10081] +++ exited with 0 +++
[pid 10080] +++ exited with 0 +++
[pid 10079] +++ exited with 0 +++
Параллельная версия (4 потоков):
Время: 2.738 мс
Ускорение: 0.656
Эффективность: 0.164
+++ exited with 0 +++
```

```
olgak@Olga:/mnt/c/capybara_my/Учеба/МАИ/2 курс/3 семестр/ОСи/Labs_OS/lab2$ strace -f -e trace=clone3 ./lab2 --size 200 --window 5 --iterations 30 --thre
ads 8
Последовательная версия: 105.993 миллисекунд

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x
791b83bff990, parent_tid=0x791b83bff990, exit_signal=0, stack=0x791b833ff000, stack_size=0x7fff80, tls=0x791b83bff6c0}strace: Process 10275 attached
 => {parent_tid=[10275]}, 88) = 10275
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b833fe990, parent_tid=0x791b833fe990, exit_signal=0, stack=0x791b82bfe000, stack_size=0x7fff80, tls=0x791b833fe6c0}strace: Process 10276
attached
 => {parent_tid=[10276]}, 88) = 10276
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b82bfd990, parent_tid=0x791b82bfd990, exit_signal=0, stack=0x791b823fd000, stack_size=0x7fff80, tls=0x791b82bfd6c0}strace: Process 10277
attached
 => {parent_tid=[10277]}, 88) = 10277
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b823fc990, parent_tid=0x791b823fc990, exit_signal=0, stack=0x791b81bfc000, stack_size=0x7fff80, tls=0x791b823fc6c0}strace: Process 10278
attached
 => {parent_tid=[10278]}, 88) = 10278
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b81bfb990, parent_tid=0x791b81bfb990, exit_signal=0, stack=0x791b813fb000, stack_size=0x7fff80, tls=0x791b81bfb6c0}strace: Process 10279
attached
 => {parent_tid=[10279]}, 88) = 10279
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b813fa990, parent_tid=0x791b813fa990, exit_signal=0, stack=0x791b80bfa000, stack_size=0x7fff80, tls=0x791b813fa6c0}strace: Process 10280
attached
 => {parent_tid=[10280]}, 88) = 10280
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b80bf9990, parent_tid=0x791b80bf9990, exit_signal=0, stack=0x791b803f9000, stack_size=0x7fff80, tls=0x791b80bf96c0}strace: Process 10281
attached
 => {parent_tid=[10281]}, 88) = 10281
[pid 10274] clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x791b803f8990, parent_tid=0x791b803f8990, exit_signal=0, stack=0x791b7fbf8000, stack_size=0x7fff80, tls=0x791b803f86c0}strace: Process 10282
attached
 => {parent_tid=[10282]}, 88) = 10282
[pid 10282] +++ exited with 0 +++
[pid 10280] +++ exited with 0 +++
[pid 10279] +++ exited with 0 +++
[pid 10281] +++ exited with 0 +++
[pid 10278] +++ exited with 0 +++
[pid 10277] +++ exited with 0 +++
[pid 10276] +++ exited with 0 +++
[pid 10275] +++ exited with 0 +++
Параллельная версия (8 потоков):
Время: 35.100 мс
Ускорение: 3.020
Эффективность: 0.377
+++ exited with 0 +++
```

Сравнительная таблица для параллельной версии (--size 500 --window 5 --iterations 150):

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	3131.506	0.956	0.956
2	1668.716	1.836	0.918
4	997.325	3.109	0.777
8	820.436	3.677	0.460
12	600.015	4.978	0.415
32	651.755	4.705	0.147
128	653.831	4.552	0.036
1024	1387.327	2.216	0.002

Эффективность закономерно снижается с ростом числа потоков, так как фактическое ускорение упирается в количество логических ядер, равное 12, в то время как параметр p продолжает увеличиваться. При p > 12 потоки физически не могут выполняться одновременно и начинают работать по очереди, конкурируя за ресурсы процессора. Из-за этого процессор тратит больше времени на переключение между потоками и ожидание синхронизации, чем на сами вычисления. Поэтому добавление новых потоков останавливает рост ускорения и начинает его снижать.

Вывод

В ходе выполнения лабораторной работы были изучены принципы многопоточного программирования в ОС Linux с использованием POSIX Threads. На практике реализован алгоритм параллельной свёртки матрицы: строки распределялись между потоками, а синхронизация на каждой итерации обеспечивалась барьером. Ускорение демонстрирует почти линейный рост при малом числе потоков, выходит на максимум в районе количества логических ядер и уменьшается при увеличении до сотен потоков из-за высоких издержек на их создание и переключение контекста, в то время как эффективность монотонно убывает. Основной сложностью стала организация корректного обмена данными между итерациями, так как результат текущего шага является входными данными для следующего. Проблема была решена использованием барьера, который гарантирует, что все потоки завершат запись в буфер до начала чтения из него на следующей итерации. Дополнительно были рассчитаны ускорение и эффективность параллельной версии, а также продемонстрировано создание потоков средствами ОС.